RUDRA GROUP PG

Master Technical Documentation Suite

Comprehensive Architecture, Database Design & API Specifications

Laravel Admin Panel & Flutter Student Mobile Companion Application

Document Version: v1.0.0 • Date: August 14, 2026

Rudra Group PG Management System - Documentation Index

Welcome to the central technical documentation for the **Rudra Group PG Management System**.

System Architecture Overview

The system operates as a unified platform supporting multi-branch Paying Guest (PG) facilities. It consists of a single Laravel backend with a single centralized MySQL database, serving a web dashboard for Super/Sub Admins and a Flutter mobile app for resident students.

```
rudragrouppg/
│
├── docs/                                  # Technical & Architectural Documentation
│   ├── README.md                          # Documentation Index (This file)
│   ├── 01_master_architecture_and_business_model.md  # Core Business Flow, Roles & Modul
│   ├── 02_database_schema_and_domain_design.md       # Central DB Schema & Relationships
│   ├── 03_flutter_student_app_specification.md      # Flutter App UI/UX & Module Specs
│   └── 04_api_contract_and_integration_guide.md      # REST API Specification (Laravel <
│
├── admin/                                 # Laravel Backend & Web Dashboard (Super Admir
│
└── student/                               # Flutter Mobile Application (Resident Compani
```

Documentation Roadmap

1. **Master Architecture & Business Model**
    - Physical visit & QR-code branch detection workflow.
    - User roles & permission boundaries (Super Admin, Sub Admin, Student).
    - Module breakdowns for Super Admin, Sub Admin, and Student.
    - Peer-to-Peer payment verification & Electricity workflow.
2. **Database Schema & Domain Design**
    - Entity-Relationship definitions and foreign key constraints.

- Branch isolation strategy (`branch_id` scoping).
- Tables: `branches` , `users` , `branch_user` , `rooms` , `beds` , `bookings` , `residents` , `payments` , `electricity_readings` , `complaints` , `notices` , `audit_logs` .

3. **Flutter Student App Specification**
   - Design system: Brand colors, Poppins typography, Material Design 3, component styling.
   - Interactive Bed Selection visual layout rules.
   - Complete specifications for all 18 Student screens.
   - Code structure & state management integration architecture.
4. **API Contract & Integration Guide**
   - REST API specifications with HTTP status codes and Sanctum Bearer token authentication.
   - Endpoint payload schemas for QR code branch resolution, room availability, bed selection, registration, booking requests, rent payment proof, and electricity readings.

Core Business Principles

- **No Public Browsing / Search**: Students do not browse PGs like Airbnb. The app is launched via QR code at the physical PG location.
- **Strict Branch Scoping**: The app renders data exclusively for the scanned branch until booking approval.
- **Peer-to-Peer Payments**: Cash or direct UPI/Bank transfer. Offline proof upload with Sub Admin manual verification.
- **Single Source of Truth**: One backend, one database, one storage API for all branches.

Document 01: Master Architecture & Business Model

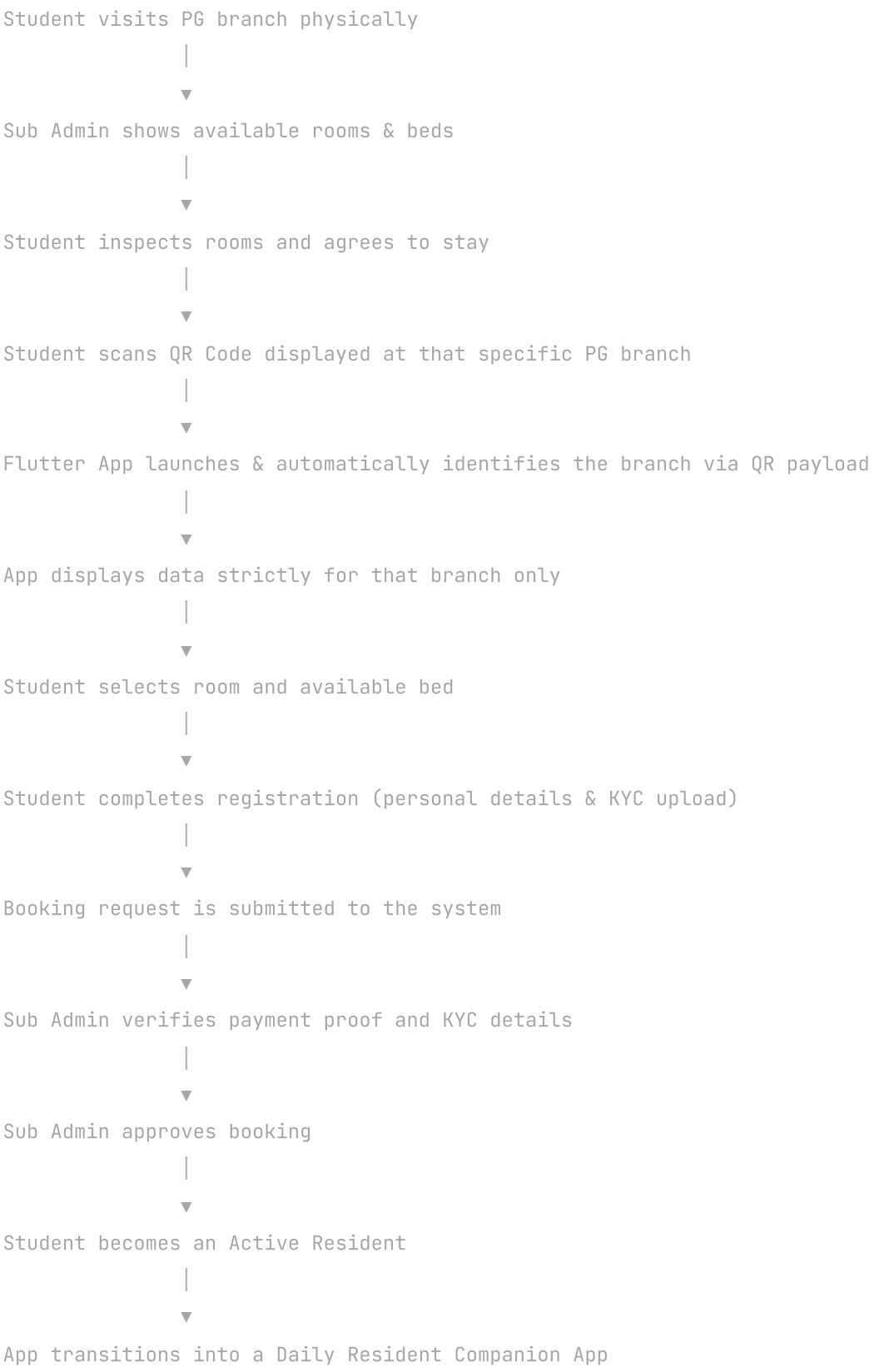
1. System Vision & Business Overview

Rudra Group PG is a multi-branch Paying Guest (PG) management platform designed to streamline operations across all PG branches under unified management.

Core Distinction: Not an E-Commerce / Listing App

This application is **NOT** a public hostel search app (e.g., Airbnb, MagicBricks, or Zolo). Students do not browse or search through hundreds of listings from home.

Actual Physical-First Business Workflow



2. Technical Architecture

The architecture enforces a single centralized backend structure to ensure data consistency, centralized reporting, and seamless scalability.

```
rudragrouppg/
├── admin/                    # Laravel Application
│   ├── Super Admin Portal    # Master control panel for all branches
│   ├── Sub Admin Portal      # Branch-level operational portal
│   ├── REST API Module       # JSON endpoints for mobile & web apps
│   └── Shared Storage        # KYC documents, payment receipts, meter photos
│
├── student/                  # Flutter Application
│   ├── Android               # Native Android build target
│   └── iOS                   # Native iOS build target
│
└── Shared Central Database   # Single MySQL relational database instance
```

Architectural Guarantees:

1. **Single Centralized Database**: All branches, users, rooms, beds, bookings, payments, and logs reside in one database.
2. **Single Unified API**: Laravel serves all API requests for both web dashboards and the Flutter mobile application.
3. **Single File Storage**: Document uploads (Aadhaar, PAN, payment receipts, meter photos) are stored in shared, structured storage.
4. **Infinite Branch Scalability**: New branches can be onboarded dynamically by Super Admin without database migration or code changes.

---

3. User Hierarchy & Roles (RBAC)

The system defines three strict user roles:

Role	Access Level	Managed System	Primary Responsibilities
Super Admin	Unlimited / Master	Laravel Web Admin	Full system oversight, branch creation, Sub Admin assignments, financial reporting, system config, global audit logs.
Sub Admin	Branch-Scoped	Laravel Web Admin	Operational management of assigned branches, bed allocation, booking approval, payment verification, electricity reading verification.
Student	Resident-Scoped	Flutter Mobile App	Bed selection, registration submission, rent & deposit tracking, electricity reading submission, complaint tracking, profile management.

---

4. Module Breakdown by User Role

A. Super Admin Modules (Laravel Web)

1. **Executive Dashboard**: Real-time revenue, overall occupancy rates, branch performance, pending booking requests, active complaints.
2. **Branch Management**: Create, update, deactivate PG branches; assign unique QR codes and branch managers.
3. **Sub Admin Management**: Create Sub Admin accounts and bind them to one or multiple PG branches.
4. **Room & Bed Master**: Configure rooms, bed counts, sharing types (1/2/3/4 sharing), rent tariffs, security deposit rates, and amenities.
5. **Student Directory**: Master database of all registered and resident students across all branches.
6. **Booking Management**: Override approvals, view booking histories, handle cancellations and room transfers.
7. **Financial & Payment Hub**: Consolidated revenue reports, deposit holding summaries, outstanding rent ledgers.
8. **Electricity Master**: Set per-unit electricity rates per branch, audit billing submissions.
9. **Reports & Analytics**: PDF/Excel exports for financial audits, occupancy metrics, payment defaults.
10. **System Settings & Audit Logs**: Global platform settings, security settings, immutable admin activity logs.

B. Sub Admin Modules (Laravel Web)

1. **Branch Dashboard**: Daily occupancy metrics for assigned branches, pending approvals, overdue payments.
2. **Student Verification**: Review incoming booking requests, verify Aadhaar/PAN documents, verify payment screenshots.
3. **Room & Bed Allocation**: Assign or transfer beds visually, flag beds under maintenance or reserved.
4. **Monthly Rent Collection**: Track rent dues, record cash payments, verify UPI transactions.
5. **Deposit Verification**: Verify initial security deposit payments before room key handover.
6. **Electricity Bill Verification**: Audit student-submitted meter readings and photo proof, approve calculated amounts.
7. **Complaint & Notice Desk**: Respond to student complaints, issue branch notices and announcements.
8. **Branch Reports**: Daily collection summaries, occupancy status reports for local branch.

C. Student Companion Modules (Flutter App)

1. **Onboarding & Branch Lock**: Splash screen, welcome flow, QR branch detection mechanism.
2. **Property Overview & Booking**: Branch details, room catalog, visual bed matrix, registration & KYC submission.
3. **Resident Portal (Post-Approval)**:
    - **My Room**: View room number, assigned bed, roommates, joining date, monthly rent rate.
    - **My Payments**: Outstanding rent, security deposit breakdown, payment proof uploader (Cash/UPI/Bank), payment receipts history.
    - **Electricity Submission**: Monthly meter reading form with camera snapshot attachment and bill history.
    - **Notifications & Announcements**: Notices from Sub Admin, rent reminders, approval alerts.
    - **Documents Vault**: View uploaded Aadhaar, PAN, and payment verification receipts.
    - **Support & Complaints**: One-tap phone/WhatsApp connection to branch manager, complaint ticketing system.

---

5. Key Operational Workflows

A. Peer-to-Peer (P2P) Payment Workflow

The system does not integrate direct online payment gateways (e.g. Razorpay/Stripe) for transaction processing. Payments are offline P2P transfers:

1. System generates payment due notice (Monthly Rent, Deposit, or Electricity).
2. Student makes offline payment (Cash to Sub Admin OR Direct UPI / Bank Transfer via displayed QR/UPI ID).
3. Student uploads transaction proof (Reference Number + Payment Screenshot) via Flutter app.
4. Sub Admin receives verification task on Laravel dashboard.
5. Sub Admin verifies funds received in bank/cash register and clicks **Approve**.
6. System updates ledger balance to Paid and generates a digital receipt.

B. Monthly Electricity Billing Workflow

1. On the 1st of every month, student receives an electricity reading submission prompt.
2. Student enters current meter reading number and attaches a live photograph of the physical meter.
3. Sub Admin audits the reading against previous month's final reading and photo proof.
4. Upon Sub Admin approval, system auto-calculates total units consumed (\times) branch unit rate.
5. Electricity bill is added to the student's monthly payment dues.

---

6. Future Scalability Roadmap

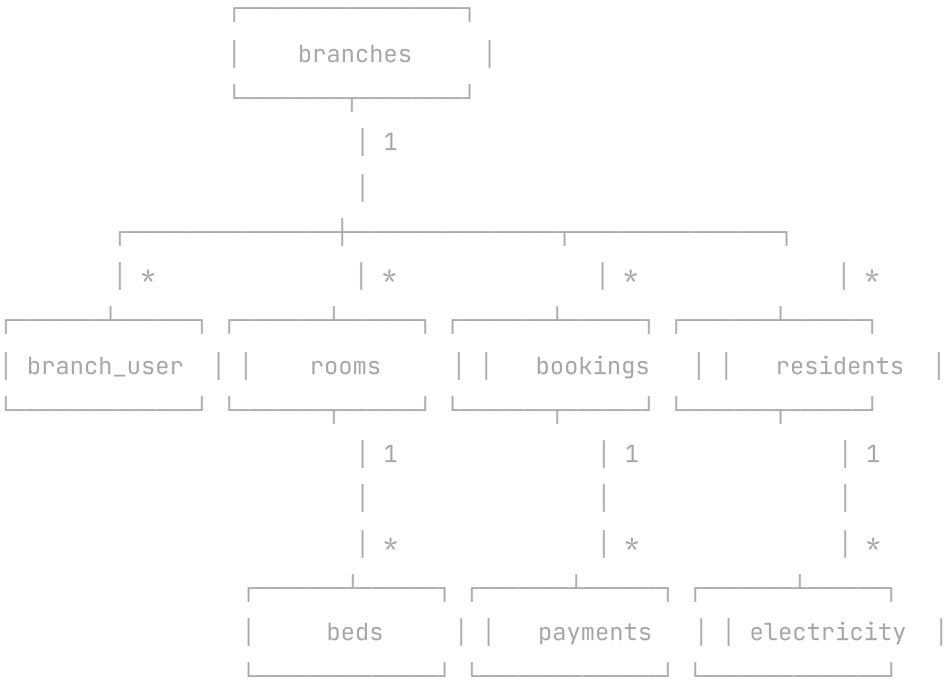
The architecture is prepared for seamless integration of advanced features without schema restructuring:

- **Biometric & Smart Door Locks**: API readiness for IoT access control based on active resident status.
- **Visitor & Leave Management**: Resident gate-pass requests and visitor log tracking.
- **Food & Laundry Management**: Daily meal opt-in/opt-out and laundry token tracking.
- **WhatsApp & Push Notifications**: Automated rent reminder triggers via WhatsApp Business API and FCM.

Document 02: Database Schema & Domain Design

1. Central Database Strategy

The **Rudra Group PG** system relies on a single MySQL database powering all PG branches. Branch isolation is achieved logically via `branch_id` scoping across tables.



---

2. Table Definitions & Relational Schema

A. Core System & Access Control

`branches`

Stores PG branch metadata and QR code identifiers.

```
CREATE TABLE branches (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    branch_code VARCHAR(50) UNIQUE NOT NULL, -- e.g., 'PG-NRD-01'
    name VARCHAR(150) NOT NULL,              -- e.g., 'Naroda Branch'
    address TEXT NOT NULL,
    city VARCHAR(100) NOT NULL DEFAULT 'Ahmedabad',
    state VARCHAR(100) NOT NULL DEFAULT 'Gujarat',
    pincode VARCHAR(10) NOT NULL,
    contact_number VARCHAR(20) NOT NULL,
    manager_name VARCHAR(100) NULL,
    qr_code_hash VARCHAR(255) UNIQUE NOT NULL, -- Matched against QR scan
    electricity_unit_rate DECIMAL(8, 2) NOT NULL DEFAULT 10.00,
    status ENUM('active', 'inactive') DEFAULT 'active',
    created_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

users

Unified table for Super Admins, Sub Admins, and Registered Students.

```
CREATE TABLE users (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(150) NOT NULL,
    email VARCHAR(150) UNIQUE NULL,
    phone VARCHAR(20) UNIQUE NOT NULL,
    password VARCHAR(255) NULL, -- Optional for quick student login
    role ENUM('super_admin', 'sub_admin', 'student') NOT NULL,
    status ENUM('active', 'inactive', 'suspended') DEFAULT 'active',
    fcm_token VARCHAR(255) NULL,
    remember_token VARCHAR(100) NULL,
    created_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

branch_user

Pivot table establishing multi-branch assignment for Sub Admins.

```
CREATE TABLE branch_user (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    user_id BIGINT UNSIGNED NOT NULL,
    branch_id BIGINT UNSIGNED NOT NULL,
    assigned_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (branch_id) REFERENCES branches(id) ON DELETE CASCADE,
    UNIQUE KEY user_branch_unique (user_id, branch_id)
);
```

B. Property Management Schema

`rooms`

Defines physical rooms inside each branch.

```
CREATE TABLE rooms (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    branch_id BIGINT UNSIGNED NOT NULL,
    room_number VARCHAR(50) NOT NULL, -- e.g., '101', '202-A'
    floor_number INT NOT NULL DEFAULT 1,
    sharing_type ENUM('1_sharing', '2_sharing', '3_sharing', '4_sharing') NOT NULL,
    monthly_rent DECIMAL(10, 2) NOT NULL,
    security_deposit DECIMAL(10, 2) NOT NULL,
    is_ac BOOLEAN DEFAULT FALSE,
    has_attached_washroom BOOLEAN DEFAULT TRUE,
    description TEXT NULL,
    status ENUM('available', 'full', 'maintenance') DEFAULT 'available',
    created_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (branch_id) REFERENCES branches(id) ON DELETE CASCADE,
    UNIQUE KEY branch_room_unique (branch_id, room_number)
);
```

`beds`

Defines individual beds within a room for visual interactive selection.

```
CREATE TABLE beds (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    room_id BIGINT UNSIGNED NOT NULL,
    bed_code VARCHAR(50) NOT NULL, -- e.g., 'Bed-A', 'Bed-B'
    status ENUM('available', 'occupied', 'reserved', 'maintenance') DEFAULT 'available',
    created_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (room_id) REFERENCES rooms(id) ON DELETE CASCADE,
    UNIQUE KEY room_bed_unique (room_id, bed_code)
);
```

C. Booking & Resident Schema

`bookings`

Stores initial student booking applications before physical bed allocation approval.

```
CREATE TABLE bookings (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    booking_number VARCHAR(50) UNIQUE NOT NULL, -- e.g., 'BK-2026-0089'
    student_id BIGINT UNSIGNED NOT NULL,
    branch_id BIGINT UNSIGNED NOT NULL,
    room_id BIGINT UNSIGNED NOT NULL,
    bed_id BIGINT UNSIGNED NOT NULL,
    full_name VARCHAR(150) NOT NULL,
    phone VARCHAR(20) NOT NULL,
    aadhaar_number VARCHAR(20) NOT NULL,
    pan_number VARCHAR(20) NULL,
    aadhaar_front_url VARCHAR(255) NOT NULL,
    aadhaar_back_url VARCHAR(255) NOT NULL,
    pan_card_url VARCHAR(255) NULL,
    status ENUM('pending', 'approved', 'rejected', 'cancelled') DEFAULT 'pending',
    rejection_reason TEXT NULL,
    processed_by BIGINT UNSIGNED NULL, -- Sub Admin user_id
    processed_at TIMESTAMP NULL,
    created_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (student_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (branch_id) REFERENCES branches(id) ON DELETE CASCADE,
    FOREIGN KEY (room_id) REFERENCES rooms(id) ON DELETE CASCADE,
    FOREIGN KEY (bed_id) REFERENCES beds(id) ON DELETE CASCADE,
    FOREIGN KEY (processed_by) REFERENCES users(id) ON DELETE SET NULL
);
```

`residents`

Maintains official active resident records after booking approval.

```
CREATE TABLE residents (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    student_id BIGINT UNSIGNED NOT NULL UNIQUE,
    booking_id BIGINT UNSIGNED NOT NULL UNIQUE,
    branch_id BIGINT UNSIGNED NOT NULL,
    room_id BIGINT UNSIGNED NOT NULL,
    bed_id BIGINT UNSIGNED NOT NULL,
    joining_date DATE NOT NULL,
    vacating_date DATE NULL,
    status ENUM('active', 'vacated', 'evicted') DEFAULT 'active',
    created_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (student_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (booking_id) REFERENCES bookings(id) ON DELETE CASCADE,
    FOREIGN KEY (branch_id) REFERENCES branches(id) ON DELETE CASCADE,
    FOREIGN KEY (room_id) REFERENCES rooms(id) ON DELETE CASCADE,
    FOREIGN KEY (bed_id) REFERENCES beds(id) ON DELETE CASCADE
);
```

D. Financial & Meter Schema

`payments`

Tracks Peer-to-Peer payment submissions and Sub Admin verification status.

```
CREATE TABLE payments (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    transaction_id VARCHAR(100) UNIQUE NOT NULL, -- e.g., 'PAY-2026-9912'
    resident_id BIGINT UNSIGNED NOT NULL,
    branch_id BIGINT UNSIGNED NOT NULL,
    payment_type ENUM('rent', 'security_deposit', 'electricity') NOT NULL,
    amount DECIMAL(10, 2) NOT NULL,
    payment_mode ENUM('cash', 'upi', 'bank_transfer') NOT NULL,
    transaction_reference VARCHAR(100) NULL, -- UPI Ref / Bank UTR
    proof_screenshot_url VARCHAR(255) NULL,
    status ENUM('pending', 'verified', 'rejected') DEFAULT 'pending',
    verified_by BIGINT UNSIGNED NULL, -- Sub Admin ID
    verified_at TIMESTAMP NULL,
    remarks TEXT NULL,
    created_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (resident_id) REFERENCES residents(id) ON DELETE CASCADE,
    FOREIGN KEY (branch_id) REFERENCES branches(id) ON DELETE CASCADE,
    FOREIGN KEY (verified_by) REFERENCES users(id) ON DELETE SET NULL
);
```

`electricity_readings`

Stores monthly student meter submissions and verified billing calculations.

```
CREATE TABLE electricity_readings (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    resident_id BIGINT UNSIGNED NOT NULL,
    room_id BIGINT UNSIGNED NOT NULL,
    reading_date DATE NOT NULL,
    meter_reading_value DECIMAL(10, 2) NOT NULL,
    meter_photo_url VARCHAR(255) NOT NULL,
    units_consumed DECIMAL(10, 2) NULL,
    total_amount DECIMAL(10, 2) NULL,
    status ENUM('submitted', 'approved', 'rejected') DEFAULT 'submitted',
    approved_by BIGINT UNSIGNED NULL,
    remarks TEXT NULL,
    created_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (resident_id) REFERENCES residents(id) ON DELETE CASCADE,
    FOREIGN KEY (room_id) REFERENCES rooms(id) ON DELETE CASCADE,
    FOREIGN KEY (approved_by) REFERENCES users(id) ON DELETE SET NULL
);
```

E. Support & System Schema

complaints

```
CREATE TABLE complaints (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    ticket_number VARCHAR(50) UNIQUE NOT NULL,
    resident_id BIGINT UNSIGNED NOT NULL,
    branch_id BIGINT UNSIGNED NOT NULL,
    category ENUM('plumbing', 'electrical', 'cleaning', 'wifi', 'other') NOT NULL,
    title VARCHAR(200) NOT NULL,
    description TEXT NOT NULL,
    status ENUM('open', 'in_progress', 'resolved', 'closed') DEFAULT 'open',
    resolution_notes TEXT NULL,
    created_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (resident_id) REFERENCES residents(id) ON DELETE CASCADE,
    FOREIGN KEY (branch_id) REFERENCES branches(id) ON DELETE CASCADE
);
```

audit_logs

```
CREATE TABLE audit_logs (
    id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    user_id BIGINT UNSIGNED NULL,
    action VARCHAR(150) NOT NULL,
    module VARCHAR(100) NOT NULL,
    ip_address VARCHAR(45) NULL,
    user_agent TEXT NULL,
    created_at TIMESTAMP NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL
);
```

---

3. Database Indexing & Performance Strategy

To maintain sub-second queries as data grows across multiple branches:

- **`branch_id` Indexing**: Composite indexes on `(branch_id, status)` across `rooms`, `bookings`, `residents`, `payments`.
- **Bed Lookup Optimization**: Unique index on `(room_id, bed_code)` ensuring instant visual bed grid rendering.
- **Fast QR Resolution**: Index on `qr_code_hash` in `branches`.

Document 03: Flutter Student Mobile Application Specification

1. Application Vision & UI/UX Principles

The **Rudra Group PG Student Mobile App** is a resident companion application designed specifically for students who have physically visited a PG property. It is designed using **Material Design 3**, focusing on elegance, clarity, soft visual cues, and intuitive navigation.

---

2. Brand Guidelines & Design System

Color Palette (Tokens)

```
Primary        #0F172A  (Deep Slate Navy - Main headers, primary actions)
Secondary      #2563EB  (Royal Blue - Interactive highlights, active tabs)
Accent         #14B8A6  (Teal - Key highlights, badges, success states)
Background     #F8FAFC  (Soft Off-White - Smooth canvas)
Card           #FFFFFF  (Pure White - Elevated container blocks)
Success        #16A34A  (Vibrant Green - Approved, Available, Paid)
Warning        #F59E0B  (Amber - Pending, Reserved, Verification)
Error          #DC2626  (Crimson Red - Occupied, Rejected, Overdue)
```

Typography System

- **Font Family**: `Poppins` (Google Fonts)
- **Scale**:
  - `Display Large` : 28pt Bold (`#0F172A`)
  - `Title Large` : 20pt SemiBold (`#0F172A`)
  - `Body Medium` : 14pt Regular (`#475569`)
  - `Caption` : 12pt Medium (`#64748B`)

Styling & Component Tokens

- **Border Radius**: `16.0` (Cards & Dialogs), `12.0` (Buttons & Form Fields)
- **Elevation / Shadows**: Soft Ambient Shadows (`BoxShadow(color: Color(0x0F000000), blurRadius: 10, offset: Offset(0, 4))`)
- **Spacing Scale**: 8px, 16px, 24px, 32px consistent padding.

---

3. Screen Specifications (18 Screens)

Screen 1: Splash Screen

- **Purpose**: App launch & branding.
- **Components**: Centered animated logo, "Rudra Group PG" text, "Premium Living Experience" tagline.
- **Behavior**: 2-second timer then auto-navigates to Welcome Screen (or Resident Dashboard if authenticated).

Screen 2: Welcome Screen

- **Purpose**: Introduce value proposition to first-time physical visitors.
- **Components**: Feature highlights (Digital Room Allocation, Easy Rent Payment, Monthly Meter Submission).
- **Primary CTA**: "Scan QR Code & Continue".

Screen 3: QR Branch Detection Screen

- **Purpose**: Simulates branch lock via QR payload scanning.
- **Components**: Simulated branch banner, "Detected Branch: Naroda Branch", branch manager contact badge, address summary.
- **Behavior**: Locks state to the detected branch; user cannot manually switch branches.

Screen 4: Branch Overview Screen

- **Purpose**: Show complete branch details to the visitor.
- **Components**: Hero image carousel, Amenity grid (High-speed Wi-Fi, RO Water, CCTV Security, Daily Housekeeping, Laundry), Branch Rules, Emergency Contact card.
- **CTA**: "Explore Available Rooms".

Screen 5: Room Listing Screen

- **Purpose**: Display rooms within the scanned branch.
- **Components**: Category filter pills (All, 2 Sharing, 3 Sharing, AC, Non-AC), Room Cards showing thumbnail image, room number, floor, rent amount, security deposit, and available bed count.

Screen 6: Room Details Screen

- **Purpose**: Deep dive into specific room features.
- **Components**: Image slider, Sharing capacity indicator, Included furniture tags (Bed, Study Table, Wardrobe), Attached washroom badge, AC indicator.
- **Primary CTA**: "Select Your Bed".

Screen 7: Interactive Bed Selection Screen (Primary UI Feature)

- **Purpose**: Visual grid layout representing physical room bed placement.
- **Visual Design**:
  - Color Legend: Green = Available, Amber = Reserved, Red = Occupied, Blue = Selected.
  - Interactive grid: Tapping an available bed triggers visual selection feedback (Blue border + tick badge).
- **Primary CTA**: "Proceed with Selected Bed (Bed 2B)".

Screen 8: Student Registration & KYC Screen

- **Purpose**: Collect resident details after bed selection.
- **Fields**: Full Name, Mobile Number, Aadhaar Number, PAN Number.
- **Upload Widgets**: Aadhaar Front Image, Aadhaar Back Image, PAN Card Image picker components.
- **CTA**: "Submit Booking Request".

Screen 9: Booking Submitted & Confirmation Screen

- **Purpose**: Confirmation state following registration.
- **Components**: Success illustration, Booking Reference ID (e.g. `BK-2026-8812`), Selected Branch, Room & Bed summary, "Pending Sub Admin Approval" status pill.
- **CTA**: "Go to Dashboard Status".

Screen 10: Resident Dashboard Screen (Main Home Tab)

- **Purpose**: Primary hub for active residents post-approval.
- **Components**:
  - Header: Resident Greeting + Active Branch Badge.
  - Quick Info Cards: My Room & Bed info, Next Rent Due Date, Outstanding Balance.
  - Quick Action Buttons: Pay Rent, Submit Meter Reading, Raise Complaint.

Screen 11: My Room Screen

- **Purpose**: Complete detail view of resident's current room assignment.
- **Components**: Branch Name, Floor Number, Room Number, Bed Code, Monthly Rent, Security Deposit held, Roommate list (if applicable).

Screen 12: Payments Screen (Payments Tab)

- **Purpose**: Financial ledger and P2P payment submission.
- **Components**:
  - Dues Summary: Monthly Rent Due, Electricity Bill Due, Security Deposit Paid.
  - Payment Method Options: Display PG Bank UPI ID & QR Code for scanning.
  - Proof Submission Form: Mode selection (Cash / UPI / Bank), Transaction UTR Number input, Screenshot File Picker.
  - Payment History List with verification status tags (Verified / Pending / Rejected).

Screen 13: Electricity Submission Module

- **Purpose**: Monthly meter reading submission.
- **Components**:
  - Previous Month Final Reading display.
  - Meter Reading Number Input field.
  - Camera Attachment Button for live meter photo upload.
  - Submission History Timeline showing past unit consumption and billed amounts.

Screen 14: Notifications Screen (Notifications Tab)

- **Purpose**: Announcements and alert logs.
- **Components**: Categorized list items (Rent Reminders, Payment Verification confirmations, Electricity Bills, Branch Announcements).

Screen 15: Profile Screen (Profile Tab)

- **Purpose**: Resident identity and settings hub.
- **Components**: Avatar photo, Full Name, Mobile Number, Emergency Contact details, Quick links to Documents, Support, Settings.

Screen 16: Documents Screen

- **Purpose**: Vault for verified identity documents and payment receipts.
- **Components**: Document cards with preview thumbnails for Aadhaar Card, PAN Card, Deposit Receipt, and Monthly Rent Receipts.

Screen 17: Support & Complaints Screen

- **Purpose**: Resident assistance portal.
- **Components**: Quick Call Sub Admin button, WhatsApp Chat shortcut, "Raise Complaint" ticket modal, FAQ expanders.

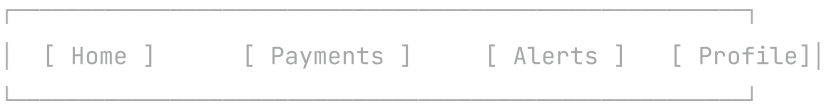
Screen 18: Settings Screen

- **Purpose**: App preferences and legal policies.
- **Components**: Push notification toggles, Terms of Service, Privacy Policy, App Version (`v1.0.0`), Logout button.

---

4. Bottom Navigation Structure

The app strictly uses a 4-tab Bottom Navigation Bar (No side drawer):



- **Home**: Resident Dashboard (or Branch Explorer if unapproved).
- **Payments**: Payment Ledger & Proof Submission.
- **Alerts**: Notifications & Announcements.
- **Profile**: Resident Profile, KYC, & Support.

---

5. Recommended Flutter Code Structure

```
lib/
├── main.dart
├── core/
│   ├── constants/        # AppColors, AppTypography, AppAssets
│   ├── theme/            # AppTheme (Material 3 configuration)
│   └── utils/            # Formatters, Validators
│
├── shared/
│   └── widgets/          # CustomButton, CustomCard, CustomTextField, StatusBadge
│
└── features/
    ├── onboarding/       # Splash, Welcome, QR Detection
    ├── booking/          # BranchOverview, RoomListing, RoomDetails, BedSelection, Regis
    ├── resident/         # Dashboard, MyRoom, Profile, Documents
    ├── payments/         # PaymentsScreen, ProofUploadModal
    ├── electricity/      # ElectricityForm, ReadingHistory
    └── support/          # SupportScreen, ComplaintModal
```

Document 04: API Contract & Integration Guide

1. Overview & Protocol Standard

This document defines the REST API contract between the **Laravel Backend** and the **Flutter Student Application**.

- **Base URL**: `https://api.rudragrouppg.com/api/v1`
- **Protocol**: HTTPS / REST
- **Authentication**: Laravel Sanctum Bearer Tokens
- **Headers**:

```
Accept: application/json
Content-Type: application/json
Authorization: Bearer <SANCTUM_TOKEN>
```

---

2. Standard Response Envelope

All API endpoints return JSON structured in a uniform envelope:

Success Response Envelope:

```
{
  "success": true,
  "message": "Operation completed successfully.",
  "data": {}
}
```

Error Response Envelope:

```
{
  "success": false,
  "message": "Validation failed or request error.",
  "errors": {
    "field_name": [
      "Error detail description."
    ]
  }
}
```

---

3. Endpoints & Payload Specifications

A. Authentication Endpoints

`POST /auth/phone-login`

- **Purpose**: Authenticate student via phone number.
- **Request Body**:

```
{
  "phone": "9876543210"
}
```

- **Response Data**:

```
{
  "success": true,
  "message": "Authentication successful.",
  "data": {
    "user_id": 42,
    "name": "Rahul Sharma",
    "phone": "9876543210",
    "token": "1|sanctum_token_string_here",
    "is_resident": true,
    "resident_status": "active"
  }
}
```

B. Branch & Room Endpoints

`GET /branches/qr/{qr_hash}`

- **Purpose**: Resolve scanned QR code hash to branch details.
- **Response Data**:

```
{
  "success": true,
  "message": "Branch identified.",
  "data": {
    "branch_id": 1,
    "branch_code": "PG-NRD-01",
    "name": "Naroda Branch",
    "address": "102, Main Highway Road, Naroda",
    "contact_number": "+91 98765 12345",
    "manager_name": "Suresh Patel",
    "electricity_unit_rate": 10.00
  }
}
```

`GET /branches/{branch_id}/rooms`

- **Purpose**: Fetch available rooms inside the locked branch.
- **Response Data**:

```
{
  "success": true,
  "data": [
    {
      "room_id": 101,
      "room_number": "101",
      "floor": 1,
      "sharing_type": "2_sharing",
      "monthly_rent": 6500.00,
      "security_deposit": 10000.00,
      "available_beds": 1,
      "total_beds": 2,
      "is_ac": true,
      "has_attached_washroom": true
    }
  ]
}
```


`GET /rooms/{room_id}/beds`

- **Purpose**: Fetch visual bed layout matrix for a selected room.
- **Response Data**:

```
{
  "success": true,
  "data": {
    "room_id": 101,
    "room_number": "101",
    "beds": [
      {
        "bed_id": 501,
        "bed_code": "Bed-101-A",
        "status": "occupied"
      },
      {
        "bed_id": 502,
        "bed_code": "Bed-101-B",
        "status": "available"
      }
    ]
  }
}
```

C. Booking & Registration Endpoints

`POST /bookings`

- **Purpose**: Submit bed booking request and upload KYC files (`multipart/form-data`).
- **Form Data Fields**:
  - `branch_id` : 1
  - `room_id` : 101
  - `bed_id` : 502
  - `full_name` : Rahul Sharma
  - `phone` : 9876543210
  - `aadhaar_number` : 123456789012
  - `pan_number` : ABCDE1234F
  - `aadhaar_front` : <BINARY_FILE>
  - `aadhaar_back` : <BINARY_FILE>
  - `pan_card` : <BINARY_FILE>

- **Response Data**:

```
{
  "success": true,
  "message": "Booking request submitted successfully.",
  "data": {
    "booking_id": 89,
    "booking_number": "BK-2026-0089",
    "status": "pending",
    "submitted_at": "2026-07-29 16:30:00"
  }
}
```

D. Resident Portal & Payment Endpoints

`GET /student/dashboard`

- **Purpose**: Retrieve active resident dashboard data post-approval.
- **Response Data**:

```
{
  "success": true,
  "data": {
    "resident_info": {
      "resident_id": 12,
      "branch_name": "Naroda Branch",
      "room_number": "101",
      "bed_code": "Bed-101-B",
      "joining_date": "2026-08-01"
    },
    "dues_summary": {
      "monthly_rent": 6500.00,
      "rent_due_date": "2026-08-05",
      "rent_status": "unpaid",
      "electricity_due": 450.00,
      "security_deposit_paid": 10000.00
    }
  }
}
```


`POST /payments/submit-proof`

- **Purpose**: Upload P2P payment screenshot proof (`multipart/form-data`).
- **Form Data Fields**:
  - `resident_id` : `12`
  - `payment_type` : `rent` // rent, security_deposit, electricity
  - `amount` : `6500.00`
  - `payment_mode` : `upi` // cash, upi, bank_transfer
  - `transaction_reference` : `UPI/123499887766`
  - `proof_screenshot` : `<BINARY_FILE>`
- **Response Data**:

```
{
  "success": true,
  "message": "Payment proof submitted for Sub Admin verification.",
  "data": {
    "transaction_id": "PAY-2026-9912",
    "status": "pending"
  }
}
```

`POST /electricity/submit-reading`

- **Purpose**: Submit monthly meter reading and camera photograph (`multipart/form-data`).
- **Form Data Fields**:
  - `resident_id` : 12
  - `room_id` : 101
  - `meter_reading_value` : 14520.50
  - `meter_photo` : <BINARY_FILE>
- **Response Data**:

```
{
  "success": true,
  "message": "Meter reading submitted successfully.",
  "data": {
    "reading_id": 45,
    "status": "submitted"
  }
}
```